COER UNIVERSITY

ROORKEE

Department of Computer Science & Engineering

LABORATORY FILE

Subject / Course	Full Stack Web Development
Experiment No.	07
Experiment Title	SQL — DDL, DML, Queries, Joins & Transactions
Student Name	Suyash Singh Tomar
Roll No.	60
CU-ID .	CU24250178
Semester	V
Faculty Name	Ms. ANKITA
Submission Date	

Academic Year 2026–27

Experiment No. 07

Title: SQL Database Lab — college_db (DDL, DML, SELECT, Joins, Subqueries, Transactions)

Aim

To create a relational database college_db with departments, students, courses and enrollments, and practice DDL, DML, SELECT queries, aggregate functions, joins, subqueries, indexes, views and transactions using the given sample data.

Theory

SQL manages relational data through DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE) and queries (SELECT). Constraints (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, NOT NULL) enforce integrity. Joins combine related tables; aggregate functions with GROUP BY/HAVING summarise data. Subqueries nest queries. Indexes speed lookups; views store named query definitions. Transactions (START TRANSACTION, COMMIT, ROLLBACK) guarantee atomic updates.

Software Required

- MySQL / MariaDB or PostgreSQL (or SQLite for practice)
- MySQL Workbench / pgAdmin / DBeaver / CLI client
- Text editor for saving .sql scripts

Procedure

1. Create database college_db and four tables as given; insert sample data; verify 8 students.
2. Part A: Create/alter/drop faculty table; note DROP vs TRUNCATE vs DELETE.
3. Part B: Insert, update, delete rows; trigger CHECK failure for age < 17.
4. Part C–D: Run SELECT filters, DISTINCT, ORDER BY, LIMIT, aggregates, HAVING.
5. Part E–F: Practice INNER/LEFT joins and subqueries (IN, EXISTS, correlated).
6. Part G: Create index and view; test TRANSACTION with ROLLBACK and COMMIT.
7. Part H (optional): RANK(), CASE grades, self-join, second-highest marks.

Database Setup & Sample Data

```
-- ============================================================
-- Lab Sheet 7 – college_db – Complete SQL Solutions
-- MySQL / PostgreSQL compatible (minor notes where needed)
-- ============================================================

CREATE DATABASE college_db;
USE college_db;   -- PostgreSQL: \c college_db

CREATE TABLE departments (     dept_id
INT PRIMARY KEY,     dept_name
VARCHAR(50) NOT NULL UNIQUE );

CREATE TABLE students (
student_id INT PRIMARY KEY,     name
VARCHAR(50) NOT NULL,     age
INT CHECK (age >= 17),     city
VARCHAR(50),     dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id) );

CREATE TABLE courses (
course_id INT PRIMARY KEY,
title     VARCHAR(80) NOT NULL,
credits   INT DEFAULT 3,
dept_id   INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id) );

CREATE TABLE enrollments (
student_id INT,
course_id  INT,     marks
INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id) );

INSERT INTO departments VALUES (1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');
INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),(108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
(201,'Data Structures',4,1),(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);
INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),(103,203,81),
```

Check: SELECT * FROM students; → 8 rows confirmed.

—

Part A DDL

A1–A5 SQL

```
CREATE TABLE faculty (
faculty_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
email VARCHAR(100) UNIQUE,
salary DECIMAL(10,2),   dept_id
INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id) );
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
ALTER TABLE faculty DROP COLUMN contact_no;
```

`DROP TABLE faculty;` DROP vs TRUNCATE vs DELETE: DROP removes the entire table (structure + data). TRUNCATE deletes all rows quickly but keeps the table structure (no WHERE). DELETE removes selected rows (can use WHERE) and can be rolled back inside a transaction.

Part B — DML

```
INSERT INTO students VALUES (109, 'Neha', 20, 'Roorkee', 1);
INSERT INTO students (student_id, name, age, dept_id) VALUES (110, 'Test', 20, 1); --
city stored as NULL
UPDATE students SET city = 'Roorkee' WHERE student_id = 105;
UPDATE enrollments SET marks = marks + 5 WHERE course_id = 203;
DELETE FROM students WHERE student_id IN (109, 110); -- B6
fails:
INSERT INTO students VALUES (111, 'Young', 15, 'Delhi', 1);
```

Error: CHECK constraint failed: age >= 17 — the CHECK (age >= 17) constraint on students.age rejects age 15.

Part C — SELECT Queries

```
SELECT * FROM students;
SELECT name, city FROM students;
SELECT * FROM students WHERE city = 'Roorkee';
SELECT * FROM students WHERE age BETWEEN 20 AND 22;
SELECT * FROM students WHERE city IN ('Roorkee', 'Dehradun');
SELECT * FROM students WHERE name LIKE 'A%';
SELECT * FROM students WHERE dept_id IS NULL;
SELECT DISTINCT city FROM students;
SELECT * FROM students ORDER BY age DESC, name ASC;
SELECT * FROM students ORDER BY age DESC LIMIT 3;
SELECT name AS "Student Name", age AS Age FROM students;
```

Sample output (C3 Roorkee, C7 NULL dept, C10 top 3 oldest)

```
Roorkee: Aarav, Meera, Rohan(after B3), Isha No
department: Vikram (dept_id NULL)
Top 3 oldest: Vikram 23, Meera 22, Diya 21
```

Part D Functions & Aggregation

```
SELECT COUNT(*) AS total_students FROM students;          -- 8
SELECT AVG(marks), MIN(marks), MAX(marks) FROM enrollments;
SELECT city, COUNT(*) FROM students GROUP BY city;
SELECT course_id, AVG(marks) FROM enrollments GROUP BY course_id;
SELECT city, COUNT(*) FROM students GROUP BY city HAVING COUNT(*) > 1;
SELECT d.dept_name, SUM(c.credits) FROM departments d
  JOIN courses c ON d.dept_id = c.dept_id GROUP BY d.dept_id, d.dept_name;
SELECT UPPER(name), LENGTH(name) FROM students;
```

—

D8: WHERE filters individual rows before GROUP BY. Aggregate results (COUNT, AVG) are not yet available at WHERE time. HAVING filters groups after aggregation, so conditions like COUNT(*) > 1 belong in HAVING.

Sample output

```
COUNT students = 8
AVG marks ≈ 79.3, MIN = 58, MAX = 95
Cities with >1 student: Dehradun (2), Roorkee (4) CSE
total credits = 12; ECE = 3; Mechanical = 3
```

Part E — Joins

```
-- E1 INNER JOIN
SELECT s.name, d.dept_name FROM students s
  INNER JOIN departments d ON s.dept_id = d.dept_id;
-- E2 LEFT JOIN (Vikram → NULL dept) SELECT
s.name, d.dept_name FROM students s
  LEFT JOIN departments d ON s.dept_id = d.dept_id;
-- E3 all departments (Civil has no students)
SELECT d.dept_name, s.name FROM departments d
  LEFT JOIN students s ON d.dept_id = s.dept_id;
-- E4 three-table join
SELECT s.name, c.title, e.marks FROM students s
  JOIN enrollments e ON s.student_id = e.student_id
  JOIN courses c ON e.course_id = c.course_id;
-- E5 Data Structures SELECT
s.name FROM students s
  JOIN enrollments e ON s.student_id = e.student_id
  JOIN courses c ON e.course_id = c.course_id
  WHERE c.title = 'Data Structures';
-- E6 not enrolled
SELECT s.name FROM students s
  LEFT JOIN enrollments e ON s.student_id = e.student_id
  WHERE e.course_id IS NULL;
-- E7 enrollments per course (incl. zero)
SELECT c.title, COUNT(e.student_id) AS enrolled FROM courses c
  LEFT JOIN enrollments e ON c.course_id = e.course_id
  GROUP BY c.course_id, c.title;
-- E8 dept stats
SELECT d.dept_name, COUNT(s.student_id), AVG(s.age) FROM departments d
  LEFT JOIN students s ON d.dept_id = s.dept_id
  GROUP BY d.dept_id, d.dept_name;
```

Key results

```
E2: Vikram shows NULL department
E3: Civil appears with no student name
E5: Aarav, Diya, Meera enrolled in Data Structures
E6: Vikram not enrolled in any course
```

Part F Subqueries

```
-- F1 above average age
SELECT * FROM students WHERE age > (SELECT AVG(age) FROM students);
-- F2 highest marks
SELECT s.name, e.marks FROM students s JOIN enrollments e
ON s.student_id = e.student_id
  WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
-- F3 courses with enrollments SELECT *
FROM courses WHERE course_id IN
  (SELECT DISTINCT course_id FROM enrollments);
-- F4 more than 2 courses
SELECT s.name FROM students s JOIN enrollments e
ON s.student_id = e.student_id
  GROUP BY s.student_id, s.name HAVING COUNT(*) > 2;
```

```
-- F5 dept with most students
SELECT d.dept_name, COUNT(s.student_id) AS cnt
FROM departments d JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(s.student_id) = (
  SELECT MAX(c) FROM (
    SELECT COUNT(*) AS c FROM students WHERE dept_id IS NOT NULL GROUP BY dept_id
  ) t);
-- F6 correlated: above course average SELECT
c.title, s.name, e.marks FROM enrollments e   JOIN
students s ON e.student_id = s.student_id
  JOIN courses c ON e.course_id = c.course_id WHERE
e.marks > (SELECT AVG(e2.marks) FROM enrollments e2
                WHERE e2.course_id = e.course_id);
```

Key results

```
F2: Meera with 95 (highest marks)
F4: Aarav enrolled in more than 2 courses
F5: CSE and ECE both have 3 students (tie for highest)
```

Part G — Indexes, Views, Transactions

```
CREATE INDEX idx_students_city ON students(city); EXPLAIN
SELECT * FROM students WHERE city = 'Roorkee';
-- Uses index SEARCH on idx_students_city instead of full table scan

CREATE VIEW student_marks AS
  SELECT s.name, c.title, e.marks
  FROM students s JOIN enrollments e ON s.student_id = e.student_id
  JOIN courses c ON e.course_id = c.course_id;
SELECT * FROM student_marks WHERE marks > 80; DROP
VIEW student_marks;
DROP INDEX idx_students_city ON students;

START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
ROLLBACK;   -- marks stays 88

START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
COMMIT;     -- marks permanently 100
```

Part H — Challenge Problems

```
-- H1 Rank list by total marks SELECT s.name,
SUM(e.marks) AS total_marks,        RANK() OVER (ORDER
BY SUM(e.marks) DESC) AS rnk
FROM students s JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name;

-- H2 Grades with CASE
SELECT student_id, course_id, marks,
CASE WHEN marks >= 90 THEN 'A'
       WHEN marks >= 75 THEN 'B'
       WHEN marks >= 60 THEN 'C'
       ELSE 'D' END AS grade
FROM enrollments;
```

```
-- H3 Same-city pairs (self join)
SELECT a.name AS student1, b.name AS student2, a.city
FROM students a JOIN students b
  ON a.city = b.city AND a.student_id < b.student_id;

-- H4 Second highest marks (no LIMIT) SELECT
MAX(marks) AS second_highest FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);  -- 91
```

H1 rank sample

```
1 Aarav 255 | 2 Meera 184 | 3 Diya 139 | ...
H4 second highest marks = 91 (after max 95)
```

Complete SQL Script

All statements for this lab are saved in lab7_all_queries.sql (setup + Parts A–H). Key sections:

```
-- ============================================================
-- Lab Sheet 7 – college_db – Complete SQL Solutions
-- MySQL / PostgreSQL compatible (minor notes where needed)
-- ============================================================

CREATE DATABASE college_db;
USE college_db;   -- PostgreSQL: \c college_db

CREATE TABLE departments (
dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL UNIQUE );

CREATE TABLE students (
student_id INT PRIMARY KEY,     name
VARCHAR(50) NOT NULL,     age
INT CHECK (age >= 17),     city
VARCHAR(50),     dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id) );

CREATE TABLE courses (
course_id INT PRIMARY KEY,
title     VARCHAR(80) NOT NULL,
credits   INT DEFAULT 3,
dept_id   INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id) );

CREATE TABLE enrollments (
student_id INT,
course_id  INT,     marks
INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id) );

INSERT INTO departments VALUES (1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');
INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),(108,'Isha',19,'Roorkee',2);
INSERT INTO courses VALUES
(201,'Data Structures',4,1),(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);
INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72),(103,203,81),
(104,201,95),(104,202,89),(105,204,58),
(106,203,74),(108,203,66);

-- Check
SELECT * FROM students;  -- 8 rows
-- ========== PART A: DDL ==========
-- A1
CREATE TABLE faculty (
faculty_id INT PRIMARY KEY,     name
VARCHAR(50) NOT NULL,     email
VARCHAR(100) UNIQUE,     salary
DECIMAL(10,2),     dept_id    INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
-- A2
ALTER TABLE faculty ADD COLUMN phone VARCHAR(15);
-- A3  (MySQL 8 / PostgreSQL)
ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
```

```
-- MySQL older: ALTER TABLE faculty CHANGE phone contact_no VARCHAR(15);
-- A4
ALTER TABLE faculty DROP COLUMN contact_no;
-- A5
DROP TABLE faculty;
-- DROP removes table structure + data permanently.
-- TRUNCATE removes all rows but keeps structure (fast, cannot WHERE).
-- DELETE removes rows (optional WHERE); can be rolled back in a transaction.

-- ========== PART B: DML ==========
-- B1
INSERT INTO students VALUES (109, 'Neha', 20, 'Roorkee', 1);
-- B2  city becomes NULL
INSERT INTO students (student_id, name, age, dept_id) VALUES (110, 'Test', 20, 1);

-- B3
UPDATE students SET city = 'Roorkee' WHERE student_id = 105;
-- B4
UPDATE enrollments SET marks = marks + 5 WHERE course_id = 203;
-- B5
DELETE FROM students WHERE student_id IN (109, 110);
-- B6  fails: CHECK (age >= 17)
-- INSERT INTO students VALUES (111, 'Young', 15, 'Delhi', 1);
-- Error: CHECK constraint failed: age >= 17
-- ========== PART C: SELECT ==========
-- C1 SELECT * FROM
students;
-- C2 SELECT name, city FROM
students;
-- C3 SELECT * FROM students WHERE city =
'Roorkee';
-- C4 SELECT * FROM students WHERE age BETWEEN 20 AND 22; --
C5 SELECT * FROM students WHERE city IN ('Roorkee',
'Dehradun'); -- C6 SELECT * FROM students WHERE name LIKE
'A%';
-- C7 SELECT * FROM students WHERE dept_id IS
NULL;
-- C8 SELECT DISTINCT city FROM
students;
-- C9 SELECT * FROM students ORDER BY age DESC,
name ASC;
-- C10 SELECT * FROM students ORDER BY age DESC
LIMIT 3;
-- C11
SELECT name AS "Student Name", age AS Age FROM students;
-- ========== PART D: Aggregation ==========
-- D1 SELECT COUNT(*) AS total_students FROM
students;
--  D2 SELECT  AVG(marks)  AS  avg_marks,  MIN(marks)  AS  min_marks,  MAX(marks)  AS  max_marks  FROM
enrollments; -- D3
SELECT city, COUNT(*) AS num_students FROM students GROUP BY city;
-- D4 SELECT course_id, AVG(marks) AS avg_marks FROM enrollments GROUP BY course_id; --
D5 SELECT city, COUNT(*) AS num_students FROM students GROUP BY city HAVING COUNT(*) >
1; -- D6
SELECT d.dept_name, SUM(c.credits) AS total_credits
FROM departments d JOIN courses c ON d.dept_id = c.dept_id
GROUP BY d.dept_id, d.dept_name;
-- D7
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length FROM students;
-- D8: WHERE filters rows BEFORE aggregation; HAVING filters groups AFTER aggregation.

-- ========== PART E: Joins ==========
-- E1
SELECT s.name, d.dept_name
FROM students s INNER JOIN departments d ON s.dept_id = d.dept_id;
-- E2  Vikram has NULL dept
SELECT s.name, d.dept_name
FROM students s LEFT JOIN departments d ON s.dept_id = d.dept_id;
-- E3  Civil has no students
SELECT d.dept_name, s.name
```

```
FROM departments d LEFT JOIN students s ON d.dept_id = s.dept_id; --
E4
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
-- E5
SELECT s.name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE c.title = 'Data Structures';
-- E6
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.course_id IS NULL;
-- E7
SELECT c.title, COUNT(e.student_id) AS enrolled
FROM courses c LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
-- E8
SELECT d.dept_name, COUNT(s.student_id) AS num_students, AVG(s.age) AS avg_age
FROM departments d LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;

-- ========== PART F: Subqueries ==========
-- F1 SELECT * FROM students WHERE age > (SELECT AVG(age) FROM
students); -- F2
SELECT s.name, e.marks
FROM students s JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
--  F3 SELECT  *  FROM  courses  WHERE  course_id  IN  (SELECT  DISTINCT  course_id  FROM
enrollments); -- F4
SELECT s.name FROM students s
JOIN enrollments e ON s.student_id = e.student_id GROUP
BY s.student_id, s.name HAVING COUNT(*) > 2;
-- F5
SELECT d.dept_name, COUNT(s.student_id) AS cnt
FROM departments d JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(s.student_id) = (
  SELECT MAX(c) FROM (
    SELECT COUNT(*) AS c FROM students WHERE dept_id IS NOT NULL GROUP BY dept_id
  ) t
);
-- F6 correlated
SELECT c.title, s.name, e.marks
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE e.marks > (
  SELECT AVG(e2.marks) FROM enrollments e2 WHERE e2.course_id = e.course_id
);

-- ========== PART G: Index, View, Transaction ========== --
G1
CREATE INDEX idx_students_city ON students(city); EXPLAIN
SELECT * FROM students WHERE city = 'Roorkee';
-- (MySQL: EXPLAIN; PostgreSQL: EXPLAIN ANALYZE)
-- G2
CREATE VIEW student_marks AS
SELECT s.name, c.title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
-- G3
SELECT * FROM student_marks WHERE marks > 80;
-- G4 DROP VIEW student_marks;
DROP INDEX idx_students_city ON students;  -- PostgreSQL: DROP INDEX idx_students_city;
-- G5
```

```
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201;
ROLLBACK;
-- marks remains 88
  --
G6
START TRANSACTION;
UPDATE enrollments SET marks = 100 WHERE student_id = 101 AND course_id = 201; COMMIT;
-- marks is permanently 100

-- ========== PART H: Challenges ==========
-- H1
SELECT s.name, SUM(e.marks) AS total_marks,
       RANK() OVER (ORDER BY SUM(e.marks) DESC) AS rnk
FROM students s JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name;
  -- H2 SELECT student_id,
course_id, marks,
  CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    WHEN marks >= 60 THEN 'C'
    ELSE 'D'
END AS grade
FROM enrollments;

-- H3
SELECT a.name AS student1, b.name AS student2, a.city
FROM students a
JOIN students b ON a.city = b.city AND a.student_id < b.student_id;

-- H4
SELECT MAX(marks) AS second_highest
FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);
```

Result

Database college_db was created with four related tables and sample data (8 students). All DDL, DML, SELECT, aggregate, join, subquery, index, view and transaction exercises were executed successfully. Constraint CHECK (age >= 17) rejected invalid inserts. JOIN queries correctly identified Vikram (no department / no enrollment) and Civil (no students). Transactions demonstrated ROLLBACK restoring marks and COMMIT making updates permanent. Optional challenges produced rank lists, letter grades, same-city pairs and second-highest marks without LIMIT.

Lab Sheet - 7 : SQL (College Database)

1. Database Setup (Given)

```
CREATE DATABASE college_db;
USE college_db;

CREATE TABLE departments (
    dept_id     INT PRIMARY KEY,
    dept_name   VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE students (
    student_id  INT PRIMARY KEY,
    name        VARCHAR(50) NOT NULL,
    age         INT CHECK (age >= 17),
    city        VARCHAR(50),
    dept_id     INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE courses (
    course_id   INT PRIMARY KEY,
    title       VARCHAR(80) NOT NULL,
    credits     INT DEFAULT 3,
    dept_id     INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
CREATE TABLE enrollments (
    student_id  INT,
    course_id   INT,
    marks       INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

2. Sample Data Inserted

```
INSERT INTO departments VALUES
  (1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');

INSERT INTO students VALUES
  (101,'Aarav',20,'Roorkee',1),
  (102,'Diya',21,'Dehradun',1),
  (103,'Kabir',19,'Haridwar',2),
  (104,'Meera',22,'Roorkee',1),
  (105,'Rohan',20,'Delhi',3),
  (106,'Sana',21,'Dehradun',2),
  (107,'Vikram',23,NULL),
  (108,'Isha',19,'Roorkee',2);

INSERT INTO courses VALUES
  (201,'Data Structures',4,1),
  (202,'Operating Systems',4,1),
  (203,'Digital Electronics',3,2),
  (204,'Thermodynamics',3,3),
  (205,'Machine Learning',4,1);

INSERT INTO enrollments VALUES
  (101,201,88),(101,202,76),(101,205,91),
  (102,201,67),(102,205,72),
  (103,203,81),(104,201,95),(104,202,89),
  (105,204,58),(106,203,74),(108,203,66);
```

3. Check: Students Table (8 rows)

```
SELECT * FROM students;
```

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Delhi	3
106	Sana	21	Dehradun	2
107	Vikram	23	NULL	NULL
108	Isha	19	Roorkee	2

A. DDL (Data Definition Language)

```
A1. CREATE TABLE faculty (...);
A2. ALTER TABLE faculty ADD phone VARCHAR(15);
A3. ALTER TABLE faculty RENAME COLUMN phone TO contact_no;
A4. ALTER TABLE faculty DROP COLUMN contact_no;
A5. DROP TABLE faculty;
```

Difference:

- **DROP:** Removes the table structure and data completely.
- **TRUNCATE:** Removes all rows but keeps the table structure.
- **DELETE:** Removes selected rows (can use **WHERE**).

B. DML (Data Manipulation Language)

```
B1. INSERT INTO students VALUES
      (109,'Neha',20,'Roorkee',1);
B2. INSERT INTO students (student_id,name,age,dept_id)
      VALUES (110,'Riya',21,1);
    Result: city = NULL (default/allowed)

B3. UPDATE students s
    LEFT student_id=105;
    WHERE course_id = 203;

B5. DELETE FROM students
    WHERE student_id IN (109,110);

B6. INSERT INTO students
      VALUES (111,'Test',15,'Roorkee',1);
    Error: Check constraint 'students_chk_1' (age >= 17)
```

C. SELECT Queries

```
C1. SELECT * FROM students;
C2. SELECT name, city FROM students;
C3. SELECT * FROM students WHERE city='Roorkee';
C4. SELECT * FROM students WHERE age BETWEEN 20
C5. SELECT * FROM students WHERE city IN BETWEEN 20 AND 22;
C6. SELECT * FROM students WHERE name LIKE + 5 WHAING
    ('Roorkee','Dehradun');
C7. SELECT * FROM students WHERE dept_id IS NULL;
C8. SELECT DISTINCT city FROM students;
C9. SELECT DISTINCT city FROM students;
C9. SELECT * FROM students ORDER BY age DESC, name ASC;
C10. SELECT * FROM students ORDER BY age DESC LIMIT 3;
C11. SELECT name AS "Student Name", age AS "Age"
     [illegible]  FROM students;
```

D. Functions & Aggregation

```
D1. SELECT COUNT(*) FROM students;
D2. SELECT AVG(marks), MIN(marks), MAX(marks)
    FROM enrollments;
D3. SELECT city, COUNT(*) FROM students GROUP BY city;
D4. SELECT course_id, AVG(marks)
    FROM enrollments GROUP BY course_id
                city HAVING COUNT(*) > 1;
D6. SELECT dept_id, SUM(credits) FROM courses
    GROUP BY dept_id;
D7. SELECT UPPER(name), LENGTH(name) FROM students;
D6. WHERE aggregation. HAVING is used to filter groups afte filters rows.
    aggregation.
```

E. Joins

```
E1. SELECT s.name, d.dept_name FROM students s
    INNER JOIN departments d ON s.dept_id = d.dept_id;
E2. SELECT s.*, d.dept_name FROM students s
    LEFT JOIN departments d ON s.dept_id = d.dept_id;
    → NULL dept: Vikram (student_id 107)

E3. SELECT d.dept_name FROM departments d
    LEFT JOIN students s ON s.dept_id = d.dept_id;
    → Shows departments with no students (if any)

E4. SELECT s.name, c.title, e.marks FROM students s
    JOIN enrollments e ON s.student_id = e.student_id
    JOIN courses c ON e.course_id = c.course_id;

E5. SELECT s.name FROM students s
    JOIN enrollments e ON s.student_id = e.student_id
    JOIN courses c ON e.course_id = c.course_id
    WHERE c.title = 'Data Structures';

E6. SELECT s.name FROM students s
    LEFT JOIN enrollments e ON s.student_id = e.student_id
    WHERE e.student_id IS NULL;

E7. SELECT c.course_id, c.title, COUNT(e.student_id) AS num_students
    FROM courses c LEFT JOIN enrollments e ON c.course_id = e.course_id
    GROUP BY c.course_id;

E8. SELECT d.dept_name, COUNT(s.student_id), AVG(s.age)
    FROM departments d LEFT JOIN students s ON d.dept_id = s.dept_id
    GROUP BY d.dept_name;
```

F. Subqueries

```
F1. SELECT * FROM students
    WHERE age > (SELECT AVG(age) FROM students);

F2. SELECT s.name, e.marks FROM enrollments e
    JOIN students s ON s.student_id = e.student_id
    WHERE e.marks = (SELECT MAX(marks) FROM enrollments);

F3. SELECT title FROM courses
    WHERE course_id IN (SELECT DISTINCT course_id FROM enrollments);

F4. SELECT s.name FROM students s
    WHERE (SELECT COUNT(*) FROM enrollments e
           WHERE e.student_id = s.student_id) > 2;

F5. SELECT d.dept_name FROM departments d
    WHERE d.dept_id = (SELECT dept_id FROM students
          GROUP BY dept_id ORDER BY COUNT(*) DESC LIMIT 1);

F6. SELECT s.name, c.title, e.marks FROM enrollments e
    JOIN courses c ON e.course_id = c.course_id
    JOIN students s ON s.student_id = e.student_id
    WHERE e.marks > (SELECT AVG(marks) FROM enrollments
                     WHERE course_id = c.course_id);
```

G. Indexes, Views & Transactions

```
G1. CREATE INDEX idx_students_city ON students(city);
    EXPLAIN SELECT * FROM students WHERE city = 'Roorkee';

    → Now uses 'idx_students_city' for faster search.

G2. CREATE VIEW student_marks AS
    SELECT s.name, c.title, e.marks
    FROM students s JOIN enrollments e ON s.student_id = e.student_id
    JOIN courses c ON e.course_id = c.course_id;

G3. SELECT * FROM student_marks WHERE marks > 80;
G4. DROP VIEW student_marks;
    DROP INDEX idx_students_city;

G5. START TRANSACTION;
    UPDATE enrollments SET marks = 100 WHERE student_id = 101
    AND course_id = 201;
    ROLLBACK:  → Marks remain 88 (unchanged)

G6. START TRANSACTION;
    UPDATE enrollments SET marks = 100 WHERE student_id = 101
    AND course_id = 201;
    COMMIT:  → Marks updated to 100 (permanent)
```

H. Challenge Problems (Optional)

1. Rank List (Total Marks)

```
SELECT s.name, SUM(e.marks) AS total_marks,
    RANK() OVER (ORDER BY SUM(e.marks) DESC) AS rnk
FROM students s JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY total_marks DESC;
```

Name	Total Marks	Rank
Meera	184	1
Aarav	167	2
Diya	139	3
Isha	66	4
Rohan	58	5

2. Grade Using CASE

```
SELECT s.name, c.title, e.marks,
  CASE
    WHEN e.marks >= 90 THEN 'A'
    WHEN e.marks >= 75 THEN 'B'
    WHEN e.marks >= 60 THEN 'C'
    ELSE 'D'
  END AS grade
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id;
```

Name	Course	Marks	Grade
Aarav	Data Structures	88	B
Meera	Operating Systems	89	B
Kabir	Digital Electronics	81	B

3. Pairs from Same City (Self Join)

```
SELECT s1.name AS Student1, s2.name AS Student2, s1.city
FROM students s1 JOIN students s2
  ON s1.city = s2.city AND s1.student_id < s2.student_id;
```

Student1	Student2	City
Aarav	Meera	Roorkee
Aarav	Isha	Roorkee
Diya	Sana	Dehradun

4. Second Highest Marks

```
SELECT MAX(marks) AS second_highest FROM (
  SELECT marks, DENSE_RANK() OVER (ORDER BY marks DESC) AS r
  FROM enrollments
) t WHERE r = 2;
```

second_highest
89